

BOCETO DE ENFOQUE

Agora como proyecto desarrollado con IA y tecnologías actuales

Propuesta para orientar la defensa hacia el proceso de desarrollo, integracion, despliegue y aprendizaje.

Indice

- Indice
- 1. Idea principal del nuevo enfoque
- 2. Que quiero demostrar con la defensa
- 3. Punto de partida del proyecto
- 4. Como he usado la IA durante el desarrollo
- 5. Como he montado la aplicacion por capas
- 6. Como he conseguido desplegarlo en cloud
- 7. Como se conectan las tecnologias entre si
- 8. Que he aprendido durante el proceso
- 9. Como lo explicaria a alguien desde cero
- 10. Limitaciones y mejoras futuras
- 11. Cierre de la defensa

Boceto personal de defensa: Agora con IA y tecnologías actuales

Este documento es para mi preparacion personal. Incluye frases y recordatorios escritos para estudiar el nuevo enfoque antes de explicarlo.

Indice

1. Idea principal del nuevo enfoque
2. Que quiero demostrar con la defensa
3. Punto de partida del proyecto
4. Como he usado la IA durante el desarrollo
5. Como he montado la aplicacion por capas
6. Como he conseguido desplegarlo en cloud
7. Como se conectan las tecnologias entre si
8. Que he aprendido durante el proceso
9. Como lo explicaria a alguien desde cero

10. Limitaciones y mejoras futuras

11. Cierre de la defensa

1. Idea principal del nuevo enfoque

No voy a cambiar el proyecto ni rehacerlo como si fuera otro trabajo distinto. Ahora sigue siendo una aplicacion funcional para gestionar practicas entre un centro educativo y empresas colaboradoras.

Lo que voy a cambiar es la forma de defenderlo.

En vez de plantearlo como:

He escrito manualmente todo este codigo desde cero.

Lo voy a plantear como:

He usado inteligencia artificial y tecnologias actuales para desarrollar, integrar, desplegar y validar una aplicacion web funcional.

La idea que quiero transmitir es que actualmente existen herramientas que permiten construir un proyecto bastante completo si se saben combinar: IA generativa, frameworks frontend y backend, contenedores, cloud, bases de datos, servicios de correo, pruebas y herramientas de supervision.

Mi defensa debe centrarse en explicar como he ido usando esas tecnologias, que papel ha tenido cada una, que problemas han aparecido y que he aprendido al montar todo el sistema.

Una frase que resume el enfoque:

Mi objetivo no es convencer de que he escrito a mano cada linea del codigo, sino demostrar como he podido llevar una idea hasta una aplicacion desplegada usando IA y tecnologias actuales.

2. Que quiero demostrar con la defensa

Con esta defensa quiero demostrar cinco cosas.

Primero, que he entendido el problema que queria resolver: la gestion de practicas entre centro educativo y empresas.

Segundo, que he sabido dividir ese problema en partes:

- portal interno para el centro;
- portal externo para empresas;

- backend con API;
- base de datos;
- correo;
- mensajería;
- despliegue cloud;
- herramienta de escritorio para supervisión.

Tercero, que he usado IA generativa como apoyo, no como una solución automática. La IA me ha ayudado a generar y corregir código, pero he tenido que probar, revisar, detectar errores y decidir que partes entraban o no en el alcance final.

Cuarto, que he aprendido a integrar tecnologías. El proyecto no es solo una página visual; tiene React, Symfony, Doctrine, PostgreSQL, Docker, Caddy, Brevo, Google Cloud y Electron.

Quinto, que he sido capaz de desplegarlo y probarlo fuera de mi ordenador, en una VM accesible por URL pública.

3. Punto de partida del proyecto

El punto de partida era una necesidad funcional: organizar mejor la relación entre un centro educativo y empresas colaboradoras para prácticas.

El flujo principal que quería representar era:

1. Una empresa se registra desde un portal externo.
2. La empresa verifica su correo y envía una solicitud.
3. El centro revisa esa solicitud desde el portal interno.
4. Si la acepta, la empresa queda activa.
5. El centro crea un convenio con esa empresa.
6. Se asigna un estudiante a una empresa y convenio.
7. Se hace seguimiento de la práctica.
8. Se puede cerrar con una evaluación final.

Esto me obliga a pensar en varias entidades principales:

- empresa;
- solicitud de empresa;
- cuenta del portal externo;
- estudiante;
- tutor académico;
- tutor profesional;
- convenio;

- asignacion;
- seguimiento;
- mensajes.

4. Como he usado la IA durante el desarrollo

La IA me ha servido como herramienta de apoyo en varias fases.

Analisis y estructura

La use para ordenar el problema y convertirlo en modulos: empresas, convenios, asignaciones, portal externo, autenticacion, documentos, mensajes y despliegue.

Generacion de codigo

La IA ayudo a generar partes de:

- controladores Symfony;
- entidades y relaciones;
- componentes React;
- formularios;
- llamadas a API;
- scripts de despliegue;
- pruebas.

Correccion de errores

Tambien la use para interpretar errores y proponer soluciones. Por ejemplo:

- errores de migraciones;
- errores 401, 403, 422 y 500;
- problemas de permisos;
- problemas con la URL publica;
- problemas de persistencia documental.

Lo importante es que la IA no ha hecho que todo funcione automaticamente. He tenido que comprobar resultados, ejecutar comandos, revisar logs, reconstruir contenedores y probar flujos reales.

Frase para defenderlo:

La IA me ha dado velocidad, pero el trabajo ha estado en dirigir el proceso, probar lo generado y entender como encajaban las piezas.

5. Como he montado la aplicacion por capas

Para entender el proyecto, lo explico por capas.

Capa de interfaz: React

React es la parte visual.

Tengo dos portales:

- portal interno: lo usa el centro;
- portal externo: lo usan las empresas.

El portal interno permite gestionar empresas, estudiantes, tutores, convenios, asignaciones y mensajes.

El portal externo permite a una empresa registrarse, enviar solicitud, ver estado, consultar informacion y comunicarse con el centro.

Capa de comunicacion: API

El frontend no accede directamente a la base de datos. Se comunica con el backend mediante una API.

Ejemplo:

Si quiero cargar empresas, React llama a `/api/empresas`.

Esa llamada devuelve JSON, que es el formato de datos que viaja entre frontend y backend.

Capa de backend: Symfony

Symfony recibe las peticiones de la API.

Por ejemplo:

- `/api/empresas` lo gestiona el controlador de empresas;
- `/api/convenios` lo gestiona el controlador de convenios;
- `/api/asignaciones` lo gestiona el controlador de asignaciones;
- `/api/portal-company/...` lo usa el portal externo.

Symfony valida los datos, aplica reglas de negocio y devuelve respuestas.

Capa de datos: Doctrine y PostgreSQL

Doctrine conecta Symfony con la base de datos.

En vez de escribir SQL para todo, el proyecto usa entidades PHP como:

- `EmpresaColaboradora`;
- `Convenio`;
- `AsignacionPractica`;
- `Estudiante`;
- `EmpresaMensaje`.

PostgreSQL guarda los datos reales en cloud.

Capa de despliegue: Docker y Google Cloud

Docker empaqueta los servicios en contenedores.

En cloud se levantan:

- contenedor de aplicacion;
- contenedor PostgreSQL;
- contenedor proxy con Caddy.

Google Cloud proporciona la VM donde se ejecuta todo.

Capa de supervision: Agora Desktop

Agora Desktop funciona como una consola tecnica para comprobar:

- estado del cloud;
- URL activa;
- logs;
- pruebas;
- conexion con la VM.

6. Como he conseguido desplegarlo en cloud

El despliegue ha sido una parte importante del proyecto porque demuestra que no se queda solo funcionando en local.

El proceso se puede explicar asi:

1. Crear una VM en Google Cloud.
2. Instalar Docker en la VM.
3. Preparar un `docker-compose.yml` para levantar varios servicios.
4. Configurar variables de entorno.
5. Construir la imagen de la aplicacion.
6. Levantar PostgreSQL.

7. Ejecutar migraciones de Symfony.
8. Crear usuarios iniciales.
9. Exponer la aplicacion con Caddy por HTTP/HTTPS.
10. Usar `nip.io` para acceder con una URL publica basada en la IP.
11. Configurar `agora.service` para que el sistema arranque automaticamente al iniciar la VM.

Frase para explicarlo:

El despliegue cloud me sirvio para pasar de una aplicacion local a un sistema accesible desde fuera, con contenedores, base de datos persistente, proxy y arranque automatico.

7. Como se conectan las tecnologias entre si

Un ejemplo claro es la gestion de empresas.

1. El usuario entra al portal interno.
2. React muestra la pantalla.
3. React llama a `/api/empresas`.
4. Symfony recibe la peticion.
5. El controlador de empresas consulta Doctrine.
6. Doctrine consulta PostgreSQL.
7. PostgreSQL devuelve los datos.
8. Symfony los transforma en JSON.
9. React recibe el JSON y actualiza la tabla.

Otro ejemplo es el registro de empresa externa.

1. La empresa entra al portal externo.
2. React muestra el formulario.
3. La empresa crea cuenta y rellena datos.
4. El frontend envia la solicitud al backend.
5. Symfony valida los datos.
6. Se crea una solicitud en base de datos.
7. Brevo envia correo de verificacion.
8. El centro aprueba o rechaza desde el portal interno.

Esto demuestra que las tecnologias no estan aisladas. Cada una cumple una funcion dentro del flujo.

8. Que he aprendido durante el proceso

He aprendido que montar una aplicacion completa no es solo hacer pantallas.

He aprendido a diferenciar:

- frontend: lo que ve el usuario;
- backend: donde esta la logica;
- API: comunicacion entre ambos;
- base de datos: donde se guarda la informacion;
- Docker: como se empaqueta y ejecuta;
- cloud: donde se publica;
- logs: donde se investigan fallos;
- pruebas: como se comprueba que algo funciona.

Tambien he aprendido que la IA puede generar codigo rapidamente, pero hay que revisarlo. Algunos errores que aparecieron fueron precisamente por no funcionar igual en local y en cloud.

Ejemplos de aprendizaje real:

- una migracion valida para SQLite puede fallar en PostgreSQL;
- una aplicacion puede arrancar pero fallar por permisos de cache;
- un formulario puede parecer correcto pero enviar datos incompletos;
- un error 401 no se debe mostrar al usuario sin contexto;
- subir documentos no es solo elegir un archivo, tambien hay que persistirlo bien;
- una VM puede cambiar de IP y afectar a la URL publica.

9. Como lo explicaria a alguien desde cero

Si tuviera que explicar a alguien como construir algo parecido desde cero, lo haria por pasos.

Paso 1: definir el problema

Primero hay que saber que se quiere resolver. En mi caso: gestionar practicas entre centro y empresas.

Paso 2: definir los usuarios

Los usuarios principales son:

- personal del centro;
- empresas externas;
- administrador tecnico.

Paso 3: definir los datos principales

Antes de programar hay que saber que datos existen:

- empresas;
- estudiantes;
- tutores;
- convenios;
- asignaciones;
- mensajes;
- documentos.

Paso 4: crear el backend

El backend define la API y la logica.

Aqui se crean entidades, controladores, validaciones y conexiones con base de datos.

Paso 5: crear el frontend

El frontend consume la API y muestra pantallas.

Aqui se crean formularios, tablas, botones, mensajes y navegacion.

Paso 6: conectar frontend y backend

React llama a Symfony mediante rutas `/api/...`.

Symfony responde con JSON.

Paso 7: probar flujos

No basta con que compile. Hay que probar:

- login;
- registro;
- crear empresa;
- aprobar solicitud;
- crear convenio;
- crear asignacion;
- enviar mensaje;
- revisar errores.

Paso 8: desplegar

Cuando funciona en local, se prepara Docker y se despliega en cloud.

Paso 9: revisar errores reales

En cloud aparecen problemas distintos:

- permisos;
- variables de entorno;
- puertos;
- base de datos;
- certificados;
- URL publica.

Paso 10: documentar

Finalmente hay que documentar que se ha conseguido, que tecnologías se han usado y que queda pendiente.

10. Limitaciones y mejoras futuras

No debo defender que todo esta al nivel de una aplicacion comercial terminada.

Puedo reconocer como futuras mejoras:

- gestion documental mas robusta;
- roles avanzados;
- multi-centro;
- backups automaticos;
- dominio propio estable;
- mas pruebas E2E;
- seguridad avanzada;
- auditoria mas completa.

Frase:

He priorizado demostrar el flujo principal y el despliegue funcional. Algunas partes quedan como evolucion futura porque llevarlas a produccion requeriria mas tiempo.

11. Cierre de la defensa

La conclusion que quiero transmitir es:

Agora demuestra como, usando IA y tecnologias actuales, he podido construir una aplicacion web funcional con portal interno, portal externo, backend, base de datos, despliegue cloud y supervision tecnica.

Tambien quiero dejar claro que he aprendido una leccion importante:

La IA ayuda a desarrollar mas rapido, pero no elimina la necesidad de entender, probar, revisar errores y explicar como funciona el sistema.

Mi defensa tiene que centrarse en ese aprendizaje: no vender el proyecto como codigo escrito manualmente al 100 %, sino como una demostracion de como se puede usar tecnologia actual para llegar a una solucion funcional real.